

Code review — andrei-incapsulare-teorie

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 2 — 2026-10-08 — sha 9318f8f

Ce s-a schimbat de la runda 1: ai scris două forme noi cu tabel — `MasiniForm` (la oră, 1 octombrie) și `CalculatoareForm` (tema, 8 octombrie) — plus clasele `Masina` și `Calculator`. Ai șters `C1form`, `Form1` și `Form2`.

Poarta de compilare: TRECUTĂ

Compilat pe **.NET Framework 4.7.2 real**, cu MSBuild, într-un Windows 11:

```
incapsulare-teorie -> bin\Debug\incapsulare-teorie.exe  
EXITCODE=0
```

Apoi am deschis ambele forme din executabilul compilat și **am apăsât butoanele din cod**, citind tabelul după fiecare apăsare. Unde scrie mai jos „reprodus”, chiar am rulat.

Ce ai făcut bine

- **B1 din runda 1 e închis.** `Application.Run` e comentat (`Program.cs:19`). Am citit executabilul: `Main` creează acum `MasiniForm`, deci aplicația pornește cu o fereastră.
- **`MasiniForm` trece testul celor trei apăsări.** Am apăsât `Incarca` de trei ori la rând: `10`, `10`, `10` rânduri. Lista se umple o singură dată, la `Load`, iar butonul doar o afișează — exact tiparul din fișă.
- **Lista e un câmp al forme** (`MasiniForm.cs:16`), nu o variabilă locală. Asta e fix lecția contorului din runda 1: ce trebuie să trăiască între două apăsări stă în clasă, nu în metodă.
- **Cele patru proprietăți de pe `dgvMasini`** — `ReadOnly`, `AllowUserToAddRows = False`, `RowHeadersVisible = False`, `AutoSizeColumnsMode = Fill` — sunt puse toate (`MasiniForm.Designer.cs:54-60`).

- **Sterge verifică CurrentRow înainte.** Am apăsăat **Sterge** pe un tabel gol: apare mesajul și metoda se oprește. Nu crapă.
- **Columns.Add(nume, antet) folosit corect** — nume tehnic **anfabricatie**, antet **An-Fabricatie**. Iar ordinea din **Rows.Add** e aceeași cu ordinea coloanelor, pe ambele forme.
- **Toate controalele au nume** — **textBoxMarca**, **btnIncarca**, **lblHP**. Niciun **button1**, niciun **textBox3**.

● Critice

B1 ● În **CalculatoareForm**, fiecare apăsare pe **Incarca** mai adaugă zece calculatoare

CalculatoareForm.cs:164 · **CalculatoareForm.Designer.cs** (lipsește linia de **Load**)

CUM E ACUM: am apăsăat **Incarca** de trei ori la rând. Reprodus:

```
apasarea 1 -> randuri=11 lista=10
apasarea 2 -> randuri=21 lista=20
apasarea 3 -> randuri=31 lista=30
```

La a treia apăsare ai 30 de calculatoare în tabel — fiecare dintre cele zece, de trei ori.

DE CE: metoda **CalculatoareForm_Load** există în cod, dar **nu e legată de formă**. O metodă devine handler doar prin linia din Designer **this.Load += ...**. La **MasiniForm** linia există (**MasiniForm.Designer.cs:224**); la **CalculatoareForm** nu există deloc. Am verificat: după deschiderea formei, lista are **0** elemente — **Load** nu s-a apelat niciodată.

Asta se întâmplă când copiezi metoda dintr-o formă în alta în loc s-o creezi cu dublu-click. Același lucru se vede și la buton: ai două metode, **btnIncarca_Click** și **btnIncarca_Click_1**. Visual Studio pune **_1** când dai dublu-click și numele există deja în fișier — adică **btnIncarca_Click** venea copiat din **MasiniForm**, și cea legată de buton e **_1**.

Apoi, cu lista goală, ai pus **LoadCalculatoare()** în butonul **_1**. Asta a umplut tabelul, dar **LoadCalculatoare** face **calculatoare.Add(...)** de zece ori **pe un câmp** — câmpul trăiește cât trăiește forma, deci la fiecare apăsare lista crește cu zece. **Rows.Clear()** golește tabelul, nu lista.

FIX: lista se umple o singură dată. Dublu-click pe suprafața formei (ca să apară linia **Load** în Designer) sau **LoadCalculatoare()** în constructor; în buton rămâne doar afișarea. Șterge

`btnIncarca_Click` (fără `_1`) — nu e legată de nimic.

B2 **Adauga** apăsă înainte de **Incarca** închide aplicația

`MasiniForm.cs:169` · `CalculatoareForm.cs:181`

CUM E ACUM: deschizi forma, completezi casetele și apeși **Adauga** fără să fi încărcat lista. Reprodus pe ambele forme:

```
InvalidOperationException: No row can be added to a
DataGridView control that does not have columns.
Columns must be added first.
```

DE CE: un `DataGridView` proaspăt nu are nicio coloană. `Rows.Add` pune valorile pe rând în coloane, iar dacă nu există niciuna, n-are unde să le pună — și aruncă. Coloanele tale apar abia în `CreateMasiniColumns()`, care se cheamă **doar din butonul Incarca**. Deci ordinea în care apasă utilizatorul butoanele decide dacă programul crapă.

Fișa spunea: `CreateColumns()` se cheamă **o singură dată, din constructor**, imediat după `InitializeComponent()`. Atunci coloanele există din prima secundă, oricare buton ar fi apăsă primul.

FIX: mută `CreateMasiniColumns()` și `CreateCalculatoareColumns()` în constructor; scoate-le din **Incarca**. Comentariile tale din `Load` (`//CreateMasiniColumns();`) arată că erai la un pas — doar că locul e constructorul.

B3 **Sterge** pe ultimul rând din **CalculatoareForm** închide aplicația

`CalculatoareForm.cs:176` · `CalculatoareForm.Designer.cs:56-60`

CUM E ACUM: încarci lista, dai click pe rândul gol de la coada tabelului și apeși **Sterge**. Reprodus:

```
InvalidOperationException: Uncommitted new row cannot be
deleted.
```

DE CE: pe `dgvCalculatoare` n-ai pus nicio proprietate din fișă, deci `AllowUserToAddRows` a rămas `True`. Atunci tabelul are mereu, la coadă, **un rând gol pentru tastat** — de-aia ai 11 rânduri și nu 10. Rândul ăla nu e un rând adevărat, e locul în care ar urma să scrie utilizatorul. Verificarea ta `CurrentRow == null` trece (rândul există), dar `RemoveAt` refuză să-l șteargă.

Pe `dgvMasini` ai pus `AllowUserToAddRows = False`, de-aia acolo nu se întâmplă. Ai aplicat regula pe o formă, nu pe amândouă. Și cu `ReadOnly` rămas `False`, utilizatorul poate scrie direct în celulele din `dgvCalculatoare`.

FIX: pe `dgvCalculatoare` aceleași patru proprietăți ca pe `dgvMasini`.

B4 ● **Adauga** primește orice — și ce primește dispare la **Incarca**

`MasiniForm.cs:169–176` · `CalculatoareForm.cs:181–188`

CUM E ACUM: reprodus pe `MasiniForm`, după `Incarca`:

```
Adauga cu casete goale      -> rand nou: [|||||]
Adauga an="trei mii" pret=-5 -> rand nou, acceptat
Incarca                    -> 10 randuri, ambele au disparut
```

DE CE: sunt două probleme, cu aceeași cauză. `Rows.Add` scrie **în tabel**, nu în listă. Tabelul e doar vitrina — copia pe care o vezi; datele adevărate stau în `List<Masina>`. La următorul `Incarca`, tabelul se reface din listă, iar ce ai pus doar în vitrină dispare.

Și pentru că nu treci prin `Masina`, nimeni nu verifică tipul. O coloană creată cu `Columns.Add(nume, antet)` primește orice obiect, deci și textul `"trei mii"` în coloana de an. Dacă ai construi o `Masina`, compilatorul te-ar opri: `anfabricatie` e `int`, iar `txtBoxAnFab.Text` e `string`. Exact acolo te-ai oprit în codul comentat de la `MasiniForm.cs:202` — `m.anfabricatie = txtBoxAnFab.;`. Lipsea conversia: `int.TryParse`.

FIX: `Adauga` verifică întâi casetele (marca și modelul nu sunt goale, anul e un întreg între 1950 și 2026), construiește o `Masina`, o pune **în listă** cu `masini.Add(m)` și reafișează tabelul cu `CreateMasiniRows()`.

● **Importante**

M1 ● **CalculatoareForm** nu se poate deschide din aplicație

`Program.cs:19`

Am citit executabilul: `Main` creează doar `MasiniForm`, iar nicio altă metodă din proiect nu creează `CalculatoareForm`. Ca s-o vezi, trebuie să schimbi `Program.cs` și să recompilezi. Același lucru e valabil pentru `UseriForm`. E constatarea M1 din runda 1, iar acum `Form1`, care trebuia să devină

meniul, a fost ștersă. Un meniu cu câte un buton pe formă (`new CalculatoareForm().Show();`) rezolvă problema pentru toate formele viitoare.

M2 🟡 Prețul e text, deci nu se poate calcula nimic cu el

Masina.cs:15 · Calculatoare.cs:15

`pret = "80.000$"` , `pret = "4000 lei"` . Cu un `string` nu poți compara (`mai ieftină decât`), ordona, aduna sau face media — adică exercițiile 7, 8, 9 și 10 din fișă. Prețul devine `double` , iar moneda se scrie în antetul coloanei: `Pret ($)` . La fel `capacitateMotor : 3.0` ca `double` , nu `"3.0L"` .

M3 🟡 Două butoane care nu fac nimic

CalculatoareForm.Designer.cs:117-129

`btnModifica` și `btnCauta` sunt pe formă, dar nu au niciun handler. Am apăsât pe amândouă: tabelul nu se schimbă, nu apare niciun mesaj. Un buton fără efect îl încurcă pe cel care folosește aplicația. Ori îl implementezi, ori îl scoți până ajungi la exercițiul lui.

🟢 Cleanups

C1 — handleri goale lăsate de dublu-click-uri: `dataGridView1_CellContentClick` (`MasiniForm.cs:132` , `CalculatoareForm.cs:157`), `UseriForm_Load` (`UseriForm.cs:84`). Le ștergi împreună cu linia lor din Designer.

C2 — `exemplu-crud/CrudForm.cs:181` : un `grpDate_Enter` gol, din click în designer pe exemplul meu. Nu strică nimic, dar exemplul nu e al tău de modificat — deschide-l, citește-l, închide-l fără să salvezi.

C3 — nume: `MasiniForm` → `MasiniForm` (ca `CalculatoareForm`), clasa `Calculator` stă în fișierul `Calculatoare.cs` (fișierul se cheamă ca și clasa), `anfabricatie` în `Masina` vs `anFabricatie` în `Calculator` . Textul de pe `groupBoxCalculatoare` e `insert_data` ; pe ecran scrie ceva omenesc, ca `Datele calculatorului` .

C4 — `MasiniForm.cs:191-207` : metoda comentată. Dacă o decommentezi, nu compilează (`txtBoxAnFab.;`). Fie o termini (B4), fie o ștergi.

C5 — mesaje de commit: `fial joi` , `final tema` . Un mesaj spune **ce** s-a schimbat: `CalculatoareForm` cu `incarca`, `adauga`, `sterge` .

Before / After

B1 — lista se umple o singură dată

Acum:

```
private void btnIncarca_Click_1(object sender, EventArgs e)
{
    LoadCalculatoare();
    CreateCalculatoareColumns();
    CreateCalculatoareRows();
}
```

Corect:

```
public CalculatoareForm()
{
    InitializeComponent();
    CreateCalculatoareColumns();
    LoadCalculatoare();
}

private void btnIncarca_Click(object sender, EventArgs e)
{
    CreateCalculatoareRows();
}
```

B2 — coloanele există de la început

Acum:

```
private void btnIncarca_Click(object sender, EventArgs e)
{
    CreateMasiniColumns();
    CreateMasiniRows();
}
```

Corect:

```
public MasiniForm()
{
    InitializeComponent();
    CreateMasiniColumns();
}

private void btnIncarca_Click(object sender, EventArgs e)
{
    CreateMasiniRows();
}
```

B4 — **Adauga** trece prin listă

Acum:

```
dgvMasini.Rows.Add(
    txtBoxMarca.Text,
    txtBoxModel.Text,
    txtBoxAnFab.Text,
    ...);
```

Corect:

```
int an;
if (!int.TryParse(txtBoxAnFab.Text, out an)
    || an < 1950 || an > 2026)
{
    MessageBox.Show("Anul trebuie sa fie intre 1950 si 2026.");
    return;
}

Masina m = new Masina();
m.marca = txtBoxMarca.Text;
m.model = txtBoxModel.Text;
m.anfabricatie = an;
...
masini.Add(m);
CreateMasiniRows();
```

Întrebări de verificare

1. `MasiniForm` și `CalculatoareForm` au aproape același cod, dar una arată tot 10 rânduri la a treia apăsare, iar cealaltă 30. Fără să te uiți în review: ce linie are una dintre forme și cealaltă nu, și în ce fișier stă linia aia?
2. După ce adaugi o mașină cu `Adauga` (așa cum e codul acum) și apeși `Incarca`, mașina dispare. Unde fusese pusă, de fapt, și de unde reface `Incarca` tabelul?
3. De ce `Rows.Add` funcționează după `Incarca`, dar aruncă excepție dacă îl apeși primul? Ce trebuie să existe în tabel înainte de primul rând?

De făcut până la runda 3

1. **B1** — `LoadCalculatoare()` o singură dată; șterge `btnIncarca_Click` necuplat.
2. **B2** — `CreateColumns()` în constructor, pe **ambele** forme.
3. **B3** — cele patru proprietăți și pe `dgvCalculatoare`.

4. **B4** — `Adauga` cu verificare + `masini.Add(m)` + reafișare, pe **ambele** forme.
5. **M2** — `pret` devine `double`, în ambele clase.
6. **M1** — o formă-meniu din care deschizi `MasiniForm`, `CalculatoareForm` și `UseriForm`.

Verificarea ta, înainte să spui „gata”: pe fiecare formă, apeși fiecare buton **de trei ori la rând**; apoi deschizi forma din nou și apeși `Adauga` și `Sterge înainte` de `Incarca`. Aplicația trebuie să rămână deschisă de fiecare dată, iar o mașină adăugată trebuie să fie încă acolo după `Incarca`.

Runda 1 — 2026-10-01 — sha 87f3ec0

Exercițiul 1 din fișa de controale (`C1form`). Poarta de compilare: .

-  **B1 închis** — `Application.Run` comentat; aplicația pornește cu o fereastră.
- — **B2** (contorul local, arăta `1` la fiecare apăsare) — `C1form` a fost ștearsă, constatarea nu se mai aplică. Ideea din spatele ei — câmp vs. variabilă locală — o folosești corect în `masini` și greșit în B1 din runda 2.
- — **B3** (eticheta care se schimba o singură dată) — `C1form` ștearsă.
-  **M1** (`Form1` fără meniu) — **redeschis ca M1 în runda 2**; `Form1` a fost ștearsă, meniul tot lipsește.
-  **M2** — `Form2` goală, ștearsă.
-  **C1-C4** — dispărute odată cu `C1form`; **C4** (mesaj de commit) — redeschis ca C5 în runda 2.

